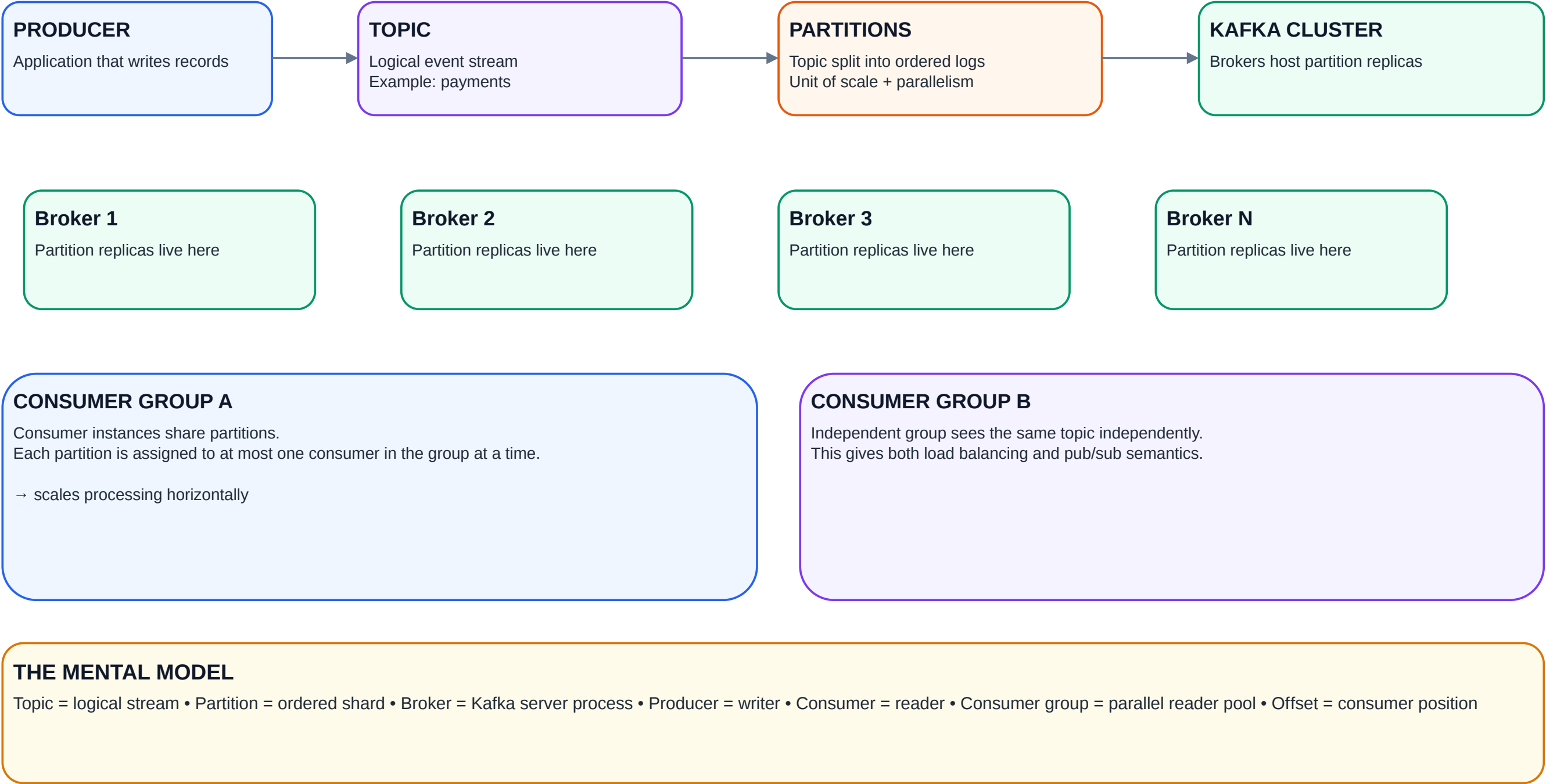


1. KAFKA — BIG PICTURE

Event streaming platform: producers append records to partitioned logs; consumers read independently at their own pace.



2. KAFKA RECORD — KNOW EVERY FIELD

A record carries data plus routing/metadata. Topic/partition/offset identify its location in the log.

VALUE

Application payload
JSON / Avro / bytes

KEY

Optional routing identity.
Same key normally maps to the same partition.

TIMESTAMP

Record timestamp.

HEADERS

Small key/value metadata.
Tracing IDs, event type, schema hints.

TOPIC

Logical stream name

PARTITION

Which ordered log shard

OFFSET

Position within that partition

JAVA PRODUCER RECORD

```
ProducerRecord<String, Payment> record =  
    new ProducerRecord<>(  
        "payments", // topic  
        "user-123", // key  
        payment // value  
    );  
  
record.headers().add(  
    "trace-id", traceId.getBytes()  
);  
  
producer.send(record);
```

WHERE DOES OFFSET COME FROM?

Kafka assigns the offset when the record is appended to a partition log.

A record's offset is unique within that partition, not globally across the topic.

Consumer position is tracked as an offset per topic-partition.

3. TOPICS + PARTITIONED LOG

Kafka persists records in append-only ordered partition logs and retains them according to topic policy.

TOPIC

Logical name / category
Multi-subscriber

PARTITION 0

0 → 1 → 2 → 3 → ...

PARTITION 1

0 → 1 → 2 → 3 → ...

PARTITION 2

0 → 1 → 2 → 3 → ...

PARTITION 3

0 → 1 → 2 → 3 → ...

LOG SEGMENTS

A partition is stored as log segments on disk. Kafka rolls segments; retention/cleaning operates on segments.

KEY PROPERTY

Append-only + replayable. Consumers do not delete messages simply by reading them. Different consumer groups can read the same records independently.

RETENTION — DELETE

`cleanup.policy=delete`

Default `retention.ms = 604800000 ms = 7 days`. Also configurable by `retention.bytes`.

Use for: events/logs where old history eventually becomes unnecessary.

LOG COMPACTION

`cleanup.policy=compact`

Keeps the latest known value for each key within a partition.

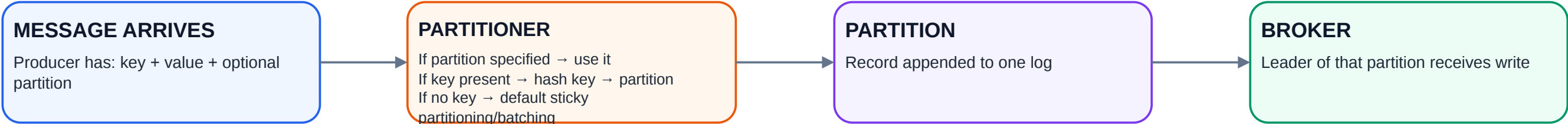
Use for: database changelogs, latest account state, cache rebuilds.

IMPORTANT

A topic can use delete, compact, or both: `cleanup.policy=delete,compact`

4. PARTITIONING — THE HEART OF KAFKA SCALE

Partitions provide storage distribution, parallel processing and per-partition ordering.



KEYED PARTITIONING
key = userId

user-123 → hash(user-123) → P2
user-456 → hash(user-456) → P0

Same key is routed consistently to the same partition under the same partitioning setup.

NO KEY
Modern Kafka’s default partitioning logic uses a sticky partition while it accumulates a batch, then switches partitions for later batches.

Do NOT memorize this as simply “random.”

ORDERING GUARANTEE
Kafka gives ordering inside a partition.

P0: A → B → C
P1: X → Y → Z

There is NO single total order across P0 + P1.

ORDERING CASE STUDY
Bank account updates:

key = accountId
→ all events for one account land in one partition
→ consumer sees that account’s events in order

Global ordering would require a single partition, sacrificing partition-level parallelism.

5. PARTITIONS = PARALLELISM + DISTRIBUTION

More partitions can increase parallelism, but they also increase metadata, file, recovery and operational cost.

1 PARTITION

Maximum consumer-group parallelism = 1
Strongest simple ordering guarantee

6 PARTITIONS

Up to 6 consumers in a group can process in parallel if assigned distinct partitions.

MANY PARTITIONS

Scales throughput, but do not pick millions casually. Partition count affects metadata, files, recovery/rebalancing and operations.

CONSUMER GROUP RULE

A topic partition can be assigned to at most one active consumer in a given consumer group at a time. Extra consumers can sit idle until partitions become available.

P0

→ C1

P1

→ C2

P2

→ C3

P3

→ C1

P4

→ C2

P5

→ C3

DESIGN QUESTION

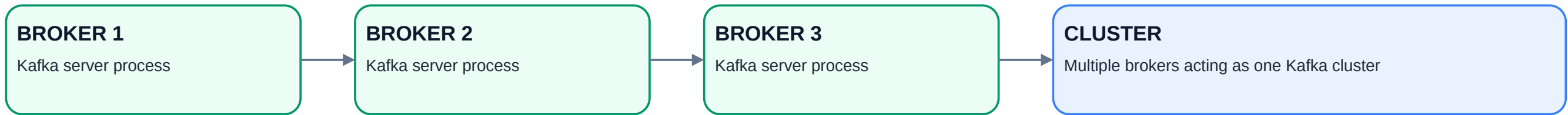
Ask: target throughput • consumer parallelism • per-partition throughput • retention volume • operational/recovery cost • future growth

ORDERING VS SCALE

More partitions = more parallelism. But ordering remains per partition. If business ordering key exists, partition by that key.

6. BROKERS + REPLICATION — HOW KAFKA SURVIVES FAILURE

A broker is a Kafka server process. Partition replicas are distributed across brokers for availability.



PARTITION P0 — REPLICATION FACTOR 3

Leader: Broker 1 ← producer writes here
Follower: Broker 2 ← copies leader log
Follower: Broker 3 ← copies leader log

Leader is the active replica for normal produce/fetch traffic.

LEADER FAILURE

If leader fails, Kafka elects another eligible replica as leader. Producers/consumers refresh metadata and continue with the new leader. Avoid unclean leader election if data loss is unacceptable.

ISR — IN-SYNC REPLICAS

ISR = replicas sufficiently caught up with the leader.

With acks=all, acknowledgements come from the in-sync replica set; min.insync.replicas can enforce a minimum ISR size.

COMMON DURABILITY SETUP

Replication factor = 3
min.insync.replicas = 2
producer acks = all

A strong common durability trade-off, subject to failure and configuration.

7. PRODUCER — HOW A WRITE REACHES KAFKA

Producer is a client library; it chooses a partition, batches records and sends produce requests to the leader.

JAVA PRODUCER

```
Properties props = new Properties();
props.put("bootstrap.servers", "broker1:9092,broker2:9092");
props.put("key.serializer", StringSerializer.class);
props.put("value.serializer", StringSerializer.class);
props.put("acks", "all");

KafkaProducer<String,String> producer =
    new KafkaProducer<>(props);

ProducerRecord<String,String> r =
    new ProducerRecord<>("orders", "order-123", "{\"amount\":100}");

producer.send(r);
producer.flush();
producer.close();
```

IMPORTANT PRODUCER TUNING

acks=all → stronger durability
retries → recover from transient errors
batch.size + linger.ms → batching efficiency
enable.idempotence=true → stronger duplicate/order handling for retries
compression.type → reduce network/storage

WRITE PATH

1. create ProducerRecord
2. partitioner chooses partition
3. producer sends to partition leader
4. leader appends + replicates
5. acknowledgement follows acks setting

KEY CHOICE

Use a business key when events for the same entity must stay ordered.

Examples: userId • accountId • orderId • deviceId

CASE STUDY — ORDER EVENTS

Producer sends:
order-created(key=123)
order-paid(key=123)
order-shipped(key=123)

Same key → same partition → partition order preserves the event sequence for that order.

8. CONSUMER — READ, PROCESS, COMMIT

Consumers pull records and control their position using offsets.

JAVA CONSUMER

```
Properties p = new Properties();
p.put("bootstrap.servers", "broker1:9092");
p.put("group.id", "payment-service");
p.put("key.deserializer", StringDeserializer.class);
p.put("value.deserializer", StringDeserializer.class);
p.put("enable.auto.commit", "false");
```

```
KafkaConsumer<String,String> c = new KafkaConsumer<>(p);
c.subscribe(List.of("payments"));
```

```
while (true) {
    ConsumerRecords<String,String> rs =
        c.poll(Duration.ofMillis(500));
    for (var rec : rs) process(rec);
    c.commitSync();
}
```

POLL MODEL

Consumer asks Kafka for records. Kafka does not push into your Java method. Consumer keeps its current position and can replay older offsets.

OFFSET

Offset = position in a partition.

Commit means: “I have processed up to this point; resume after this when appropriate.”

Offsets are tracked for consumer groups.

PROCESSING SEMANTICS

Commit AFTER processing → commonly at-least-once style; crash before commit can replay.

Commit BEFORE processing → may lose work on crash.

Exactly-once needs additional transactional/idempotent design, not just consumer commit.

PRACTICAL RULE

Make consumers idempotent where possible.

Example: eventId stored in DB as processed. A replay becomes harmless instead of applying business logic twice.

9. CONSUMER GROUPS — LOAD BALANCING + PUB/SUB

Same group = load balancing; different groups = independent subscriptions.

TOPIC: PAYMENTS

P0 P1 P2 P3

GROUP A — PAYMENT SERVICE

C1 + C2

P0

→ C1

P1

→ C1

P2

→ C2

P3

→ C2

GROUP B — ANALYTICS

C3 + C4

P0

→ C3

P1

→ C3

P2

→ C4

P3

→ C4

RULES

Same group → partitions are divided among consumers.

Different groups → each group receives its own view of the topic.

A group cannot actively process more partitions in parallel than the topic has.

REBALANCE

When consumers join/leave/fail, partition ownership can change.

During rebalancing, processing can pause depending on the protocol/application.

Design for duplicate processing after crashes.

10. RETENTION vs COMPACTION — DIFFERENT PROBLEMS

Retention = how long old history remains. Compaction = latest state per key.

DELETE RETENTION

`cleanup.policy=delete`

Default `retention.ms` = 7 days. Can also bound by `retention.bytes`.

Use for temporal data: logs, clicks, telemetry.

LOG COMPACTION

`cleanup.policy=compact`

Retain the latest known value for each key within a partition.

Use for database changelogs, latest account state, caches.

COMPACTION EXAMPLE

```
key=user-1 value=Delhi
key=user-2 value=Pune
key=user-1 value=Mumbai
```

After compaction:
user-1 → Mumbai
user-2 → Pune

Consumers can rebuild latest state
without retaining every old update.

TOMBSTONE

A record with a key and null value can represent deletion in a compacted topic. Tombstones are retained for a configurable period before they can be removed.

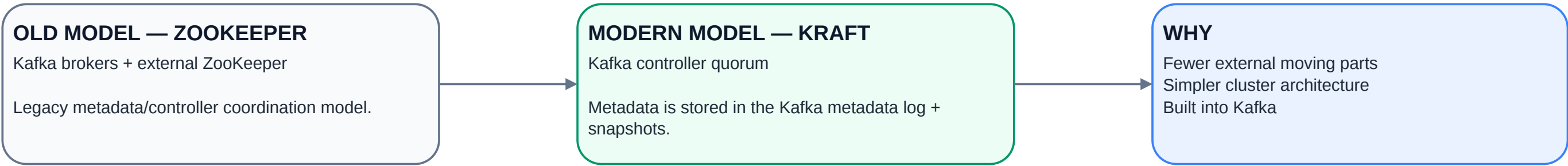
BOTH TOGETHER

`cleanup.policy=delete,compact`

Useful when you want latest state while also bounding the age/size of old segments.

11. KRAFT — MODERN KAFKA METADATA MANAGEMENT

Modern Kafka uses a built-in metadata quorum instead of ZooKeeper. Kafka 4.x docs describe KRaft controllers as the metadata holders.



KRAFT COMPONENTS

Controller quorum → manages cluster metadata
Broker → serves client data traffic
Controller(s) → metadata leadership / quorum

For critical deployments, separate controller and broker roles are recommended rather than combined mode.

FAILURE MODEL

A controller quorum needs a majority to make progress.

With $2N+1$ controllers, quorum can tolerate N controller failures without losing quorum availability.

VERSION NOTE

Kafka 4.2/4.3-era documentation uses KRaft as the current metadata architecture. Learn KRaft first for new systems; know ZooKeeper mainly for legacy clusters/migrations.

12. DURABILITY — REPLICATION, ISR, ACKS

Learn replication factor, leader/followers, ISR, acks and min.insync.replicas together.

REPLICATION FACTOR = 3

P0:

Leader → Broker1

Follower → Broker2

Follower → Broker3

Three copies of the partition log.

ISR

In-sync replicas are sufficiently caught up with the leader. ISR can shrink when followers fall behind.

ACKS

acks=0 → no broker ack wait

acks=1 → leader ack

acks=all/-1 → all ISR must acknowledge

Combine with min.insync.replicas for stronger durability.

COMMON SETUP

RF=3

min.insync.replicas=2

producer acks=all

A write can succeed when the current ISR satisfies these conditions.

FAILURE EXAMPLE

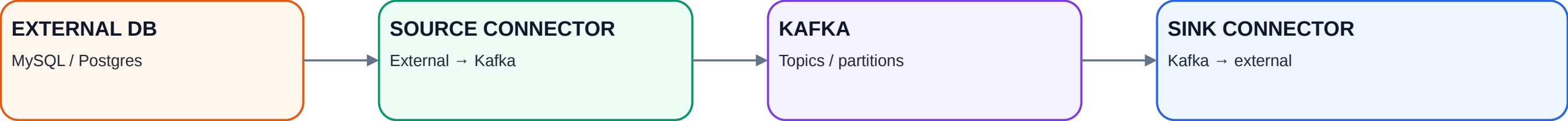
Broker1 is leader and fails. An eligible replica can become the new leader. Producer refreshes metadata and sends new writes to the new leader.

IMPORTANT

Replication does not mean every replica is always equally caught up. Learn ISR + leader election + acks together to reason about durability.

13. KAFKA CONNECT — EXTERNAL INTEGRATIONS

Kafka Connect is Kafka’s framework for moving data between Kafka and external systems without custom integration code for every source/sink.



CONNECTOR
Defines integration logic and splits work into tasks. SourceConnectors import data; SinkConnectors export data.

TASKS
Actual parallel workers doing the copying. A connector may create multiple tasks subject to tasks.max and source/sink limits.

WORKERS
Kafka Connect worker = process/node running connectors + tasks.

Distributed mode uses multiple workers for balancing, scaling and fault tolerance.

CONVERTERS
Convert between Kafka Connect’s internal record model and serialized formats. Common formats: JSON, Avro, Protobuf.

STANDALONE
Single process. Simple for development/testing or simple deployments.

DISTRIBUTED
Multiple workers. Automatic balancing, scaling and fault tolerance. Config/status/offset state is stored in Kafka topics.

14. CASE STUDY — DATABASE → KAFKA → SEARCH INDEX

Use Connect when integration is mostly configuration rather than complex business logic.

JDBC SOURCE — EXAMPLE SHAPE

```
{
  "name": "orders-source",
  "connector.class": "...JdbcSourceConnector",
  "connection.url": "jdbc:postgresql://db/orders",
  "mode": "timestamp",
  "timestamp.column.name": "updated_at",
  "topic.prefix": "db.orders.",
  "tasks.max": "4"
}
```

FLOW

Postgres → Source Connector → Kafka topic

Kafka becomes the durable streaming layer between the database and downstream systems.

END TO END

Postgres → Source Connector → Kafka → Sink Connector → Elasticsearch

Benefits: decoupling, replay, buffering, independent scaling.

SINK CONNECTOR — EXAMPLE SHAPE

```
{
  "name": "orders-search",
  "connector.class": "...ElasticsearchSinkConnector",
  "topics": "db.orders.orders",
  "connection.url": "http://elasticsearch:9200",
  "tasks.max": "4"
}
```

WHEN NOT TO USE CONNECT

Use custom application code when you need complex business logic, joins, transactional workflows, custom APIs, or domain-specific error handling that the connector cannot model cleanly.

INTERVIEW TIP

Connector ≠ worker ≠ task. Connector defines the job; workers run it; tasks perform parallel pieces.

15. KAFKA — ONE-PAGE REVISION CARD

Everything should map back to these concepts.

EVENT / RECORD

value • key • timestamp • headers

TOPIC

Logical stream

PARTITION

Ordered log + parallelism

OFFSET

Position within partition

PRODUCER

Chooses partition
Batches + sends to leader
Key often determines partition

BROKER

Server process
Hosts replicas
Leader handles normal traffic

REPLICATION

Leader + followers
ISR
Leader election

CONSUMER

Poll + process + commit offset

CONSUMER GROUP

Same group → load balancing across partitions
Different groups → independent subscription
Max active consumer parallelism is bounded by partition count

RETENTION / COMPACTION

delete → old history by time/size
compact → latest value per key
Both can be combined

KRAFT

Modern metadata quorum; ZooKeeper is legacy for older deployments.

KAFKA CONNECT

Source: external → Kafka
Sink: Kafka → external

16. SCENARIO QUESTIONS — REASON, DON'T MEMORIZE

Use these to validate real understanding.

Need order per user

key=userId → all user events map to one partition

Two services need every event

Use two different consumer groups

Leader fails

Eligible replica can become new leader

DB → Kafka quickly

Kafka Connect Source

Traffic spikes

Check consumer lag, partition/consumer capacity, downstream limits

Need 20-way parallel consumers

Topic needs at least 20 partitions for that group

Crash after process, before commit

Replay can happen → idempotent processing

Need latest account state

Compacted topic keyed by accountId

Kafka → Elasticsearch

Kafka Connect Sink

Need global ordering

Single-partition topic; lose partition-level parallelism

17. FINAL KAFKA MENTAL MODEL

Reconstruct Kafka from immutable logs + partitioning + replication + independent consumers.

1. PRODUCE

event

2. PARTITION

key → partition

3. APPEND

leader log

4. REPLICATE

followers / ISR

5. RETAIN

delete / compact

6. POLL

consumer reads

7. COMMIT

offset progress

8. REPLAY

seek old offset

THE SINGLE SENTENCE

Kafka is a distributed, partitioned append log where producers write records, brokers replicate partition logs, and consumer groups read independently by tracking offsets.

THE THINGS TO KNOW

Event → Topic → Partition → Broker → Replication/ISR → Producer → Consumer Group/Offset → Retention/Compaction → KRaft → Connect